Genius Network: Callum EDA

AUTHOR
Callum Magarian

Objectives

This is an Exploratory Data Analysis which will investigate the structure of Genius dataset used in previous Cornell research: *Expertise and Dynamics within Crowdsourced Musical Knowledge Curation: A Case Study of the Genius Platform* by Derek Lim and Austin R. Benson.

Explaining Data Sources

For the time being, our data is ran locally. This a large collection of data, but it will be computationally sufficient for the time being.

```
# connecting path
data_dir <- path.expand("~/Downloads/genius-expertise")
stopifnot(dir.exists(data_dir))
# providing basic information
paths <- list.files(data_dir, full.names = TRUE)
info <- file.info(paths)
# maintaining directory
keep <- !is.na(info$isdir) & !info$isdir
inventory <- data.frame(
  file = basename(paths[keep]),
  size_MB = round(info$size[keep] / 1024^2, 2)
)
inventory[order(inventory$size_MB, decreasing = TRUE), ]
```

```
                 file size_MB
1 annotation_info.json 1147.43
2     artist_info.json  127.33
4            lyrics.jl  107.63
8       user_info.json   83.00
7       song_info.json   51.71
3          follows.csv   44.73
5 map_artist_names.csv    0.17
6            README.md    0.00
```

The csv files themselves are not too large so these will get a simple, brief overview.

```
# tracking csv files
csv_files <- list.files(
  data_dir,
  pattern = "\\.csv$",
  full.names = TRUE,
```

```
  ignore.case = TRUE
)
basename(csv_files)
```

```
[1] "follows.csv"          "map_artist_names.csv"
```

```
# previewing csv
csv_preview <- data.table::fread(csv_files[1], nrows = 1000)
# overviewing information
names(csv_preview)
```

```
[1] "follower"  "following"
```

```
str(csv_preview)
```

```
Classes 'data.table' and 'data.frame':  1000 obs. of  2 variables:
 $ follower : chr  "CalebMac116" "deadbby" "Michael" "Jezzaboii" ...
 $ following: chr  "QueenOfCoplaints" "QueenOfCoplaints" "QueenOfCoplaints"
"QueenOfCoplaints" ...
 - attr(*, ".internal.selfref")=<pointer: 0x10391bc10>
```

```
head(csv_preview)
```

```
        follower        following
          <char>           <char>
1:   CalebMac116 QueenOfCoplaints
2:       deadbby QueenOfCoplaints
3:       Michael QueenOfCoplaints
4:     Jezzaboii QueenOfCoplaints
5:    YABOYNICKY QueenOfCoplaints
6: GnarlingtonTX QueenOfCoplaints
```

```
# same for second csv
csv_preview <- data.table::fread(csv_files[2], nrows = 1000)
names(csv_preview)
```

```
[1] "blackthought"  "Black-thought"
```

```
str(csv_preview)
```

```
Classes 'data.table' and 'data.frame':  1000 obs. of  2 variables:
 $ blackthought : chr  "PushaT" "RobMarkman" "ChanceTheRapper" "KendrickLamar" ...
 $ Black-thought: chr  "Pusha-t" "Rob-markman" "Chance-the-rapper" "Kendrick-lamar" ...
 - attr(*, ".internal.selfref")=<pointer: 0x10391bc10>
```

```
head(csv_preview)
```

```
      blackthought     Black-thought
             <char>            <char>
1:          PushaT           Pusha-t
2:      RobMarkman       Rob-markman
3: ChanceTheRapper Chance-the-rapper
4:   KendrickLamar    Kendrick-lamar
5:    Savan_DePaul      Savan-depaul
6:        camjohns       Cam-johnson
```

The csv files in general are quite simple. follows.csv covers users and who they are following. map_artist_names contains the initial musician / band's spelling and then a normalized version using dashes and first letter capitals for consistency (e.g. blackthought -> Black-thought).

Json overviews should be cautious given these file sizes are larger than the csvs, especially for annotation information.

```
json_files <- list.files(
  data_dir,
  pattern = "\\.json$",
  full.names = TRUE,
  ignore.case = TRUE
)

basename(json_files)
```

```
[1] "annotation_info.json" "artist_info.json"     "song_info.json"
[4] "user_info.json"
```

```
# providing 2000 bit prefixes for json files
peek_json <- function(path, n_bytes = 2000L) {
  con <- file(path, open = "rb")
  on.exit(close(con))

  prefix <- readBin(con, what = "raw", n = n_bytes)

  cat("\n---", basename(path), "---\n")
  cat(rawToChar(prefix), "\n")
}

for (path in json_files) {
  peek_json(path)
}
```

```
--- annotation_info.json ---
{"time": "Jan 31, 2014 11:48:16 AM", "type": "reviewed", "votes": 55, "pyongs": 0,
"acceptor": null, "url": "https://genius.com/2739115", "contribution_stats": [{"name":
"PiecesOfAMan", "contribution": 73.73}, {"name": "Incilin", "contribution": 26.27}],
"edits_lst": [{"name": "https://genius.com/Incilin", "time": "Aug 5, 2016 4:44:24 PM",
```

"content": " This woozy, atmospheric beat, has hard drums that are complemented by Kendrick\u2019s lyrics and tweaks on his vocals. \n\n Producer T-Minus and TDE\u2019s in-house engineer MixedByAli talked about the process of making this song in an interview with Complex, \n\n T-Minus: \n\n<blockquote>\n I\u2019m good friends with the A&Rs at Interscope. We were working together and he had a bunch of beats of mine. He played it for Kendrick and he just fell in love with that one beat. Its funny, I was thinking maybe an R&B artist was gonna jump on that beat. \n\n He had the beat, he went in on it, and he sent it back to me. I did a little bit of post-production. On his end, he had his people touch up things on the record, vocally and instrumentally. That\u2019s one thing that\u2019s really important, getting that post-production done because a lot of times producers just wanna send out beats and hope that the record\u2019s just gonna make itself. But it takes more than that. \n</blockquote>\n\n MixedByAli: \n\n<blockquote> That song was actually fun to mix. Me and Dr. Dre mixed that one. The beat is real open so you can kind of do whatever you want to it, as far as messing with the vocals. So I was just playing with some delays, some reverbs, tweak it out make it sound real eerie. </blockquote>\n\n On the extended version, the production switches into soft synths and then building up with the drums hitting harder in an almost boom bap st

--- artist_info.json ---
{"url_name": "Mike-bozzi", "followers": 15, "roles": [], "iq": 0, "songs": ["Post-malone-rockstar-lyrics", "Kendrick-lamar-dna-lyrics", "Childish-gambino-this-is-america-lyrics", "Kendrick-lamar-xxx-lyrics", "Kendrick-lamar-love-lyrics", "Post-malone-congratulations-lyrics", "Post-malone-psycho-lyrics", "Schoolboy-q-collard-greens-lyrics", "Kendrick-lamar-loyalty-lyrics", "Kendrick-lamar-element-lyrics", "Post-malone-i-fall-apart-lyrics", "Jay-rock-kendrick-lamar-future-and-james-blake-kings-dead-lyrics", "Tyler-the-creator-earfquake-lyrics", "Kendrick-lamar-fear-lyrics", "Post-malone-better-now-lyrics", "Nf-let-you-down-lyrics", "Kendrick-lamar-pride-lyrics", "Tyler-the-creator-see-you-again-lyrics", "Post-malone-go-flex-lyrics", "Post-malone-stay-lyrics", "Schoolboy-q-studio-lyrics", "Kendrick-lamar-feel-lyrics", "Kendrick-lamar-lust-lyrics", "Lana-del-rey-love-lyrics", "Kendrick-lamar-yah-lyrics", "The-weeknd-and-kendrick-lamar-pray-for-me-lyrics", "Kendrick-lamar-god-lyrics", "Kendrick-lamar-blood-lyrics", "Schoolboy-q-man-of-the-year-lyrics", "Mac-miller-self-care-lyrics", "Post-malone-deja-vu-lyrics", "Tyler-the-creator-911-mr-lonely-lyrics", "Schoolboy-q-that-part-lyrics", "Schoolboy-q-2-chainz-and-saudi-x-lyrics", "Post-malone-goodbyes-lyrics", "Post-malone-paranoid-lyrics", "Post-malone-over-now-lyrics", "Post-malone-no-option-lyrics", "Mac-miller-2009-lyrics", "Nf-the-search-lyrics", "Tyler-the-creator-garden-shed-lyrics", "Tyler-the-creator-new-magic-wand-lyrics", "Tyler-the-creator-i-think-lyrics", "Miguel-come-through-and-chill-lyrics", "Post-malone-rich-and-sad-lyrics", "Tyler-the-creator-and-a-ap-rocky-potato-salad-lyrics", "Sob-x-rbe-paramedic-lyrics", "Snoop-dogg-drop-it-like-its-hot-lyrics", "Nf-lie-lyrics", "Tyler-the-creator-gone-gone-thank-you-lyrics", "Post-malone-take-what-you-want-lyrics", "Schoolboy-q-break-the-bank-lyrics", "Tyler-the-creator-a-boy-is-a-gun-lyrics", "Post-malone-92-explorer-lyrics", "Post-malone-ball-for-me-lyrics", "Tyler-

--- song_info.json ---
{"url_name": "Kendrick-lamar-swimming-pools-drank-lyrics", "title": "Swimming Pools (Drank)", "primary_artist": "Kendrick-lamar", "release_date": "July 31, 2012", "pyongs": 894, "contributors": 403, "has_bio": true, "views": 5589280, "tags": ["Trap", "Conscious Hip-Hop", "Memes", "West Coast", "Rap", "Producer"], "annotations": null}

{"url_name": "Kendrick-lamar-money-trees-lyrics", "title": "Money Trees", "primary_artist": "Kendrick-lamar", "release_date": "October 22, 2012", "pyongs": 880, "contributors": 394, "has_bio": true, "views": 4592003, "tags": ["Conscious Hip-Hop", "West Coast", "Rap", "Producer"], "annotations": null}
{"url_name": "Kendrick-lamar-xxx-lyrics", "title": "XXX.", "primary_artist": "Kendrick-lamar", "release_date": "April 14, 2017", "pyongs": 188, "contributors": 389, "has_bio": true, "views": 4651514, "tags": ["Conscious Hip-Hop", "Boom Bap", "Pop", "West Coast", "Rap"], "annotations": null}
{"url_name": "A-ap-rocky-fuckin-problems-lyrics", "title": "Fuckin\u2019 Problems", "primary_artist": "A-ap-rocky", "release_date": "October 24, 2012", "pyongs": 706, "contributors": 437, "has_bio": true, "views": 7378309, "tags": ["Gangsta Rap", "Dirty South", "Atlanta", "Posse Cut", "West Coast", "Canada", "East Coast", "Rap"], "annotations": null}
{"url_name": "Kendrick-lamar-dna-lyrics", "title": "DNA.", "primary_artist": "Kendrick-lamar", "release_date": "April 14, 2017", "pyongs": 555, "contributors": 570, "has_bio": true, "views": 5113687, "tags": ["Politics", "Producer", "News", "Conscious Hip-Hop", "Trap", "West Coast", "Rap"], "annotations": null}
{"url_name": "Kendrick-lamar-poetic-justice-lyrics", "title": "Poetic Justice", "primary_artist": "Kendrick-lamar", "release_date": "October 22, 2012", "pyongs": 387, "contributors": 368, "has_bio": true, "views": 4137688, "tags": ["Conscious Hip-Hop", "West Coast", "Rap"], "annotations": null}
{"url_name": "Kendrick-lamar-bitch-dont-kill-my-vibe-lyrics", "title": "Bitch, Don\u2019t Kill My Vibe", "pr

--- user_info.json ---
{"url_name": "QueenOfCoplaints", "iq": 446264, "stats": {"Annotations": 5072, "Suggestions": 487, "Transcriptions": 1808, "Questions": 13, "Answers": 24, "Pyongs": 5601}, "roles": ["Editor", "Contributor"], "followers_num": 600, "following_num": 1}
{"url_name": "Michael", "iq": 500941, "stats": {"Annotations": 6256, "Suggestions": 2651, "Transcriptions": 332, "Questions": 73, "Answers": 54, "Pyongs": 5692}, "roles": ["Staff", "Moderator", "Editor", "Contributor"], "followers_num": 1814, "following_num": 1437}
{"url_name": "CalebMac116", "iq": 218302, "stats": {"Annotations": 1343, "Suggestions": 583, "Transcriptions": 426, "Questions": 90, "Answers": 467, "Pyongs": 309}, "roles": ["Moderator", "Editor", "Contributor"], "followers_num": 395, "following_num": 451}
{"url_name": "Jezzaboii", "iq": 4504, "stats": {"Annotations": 68, "Suggestions": 3, "Transcriptions": 2, "Questions": 0, "Answers": 0, "Pyongs": 27}, "roles": ["Contributor"], "followers_num": 39, "following_num": 30}
{"url_name": "dalmo", "iq": 180498, "stats": {"Annotations": 4513, "Suggestions": 601, "Transcriptions": 2174, "Questions": 3, "Answers": 0, "Pyongs": 1975}, "roles": ["Editor", "Contributor"], "followers_num": 657, "following_num": 590}
{"url_name": "Ezzo", "iq": 327742, "stats": {"Annotations": 5756, "Suggestions": 257, "Transcriptions": 1768, "Questions": 18, "Answers": 25, "Pyongs": 1086}, "roles": ["Contributor"], "followers_num": 394, "following_num": 341}
{"url_name": "deadbby", "iq": 189255, "stats": {"Annotations": 1708, "Suggestions": 534, "Transcriptions": 770, "Questions": 4, "Answers": 11, "Pyongs": 2236}, "roles": ["Contributor"], "followers_num": 495, "following_num": 536}
{"url_name": "GnarlingtonTX", "iq": 265140, "stats": {"Annotations": 1011, "Suggestions": 128, "Transcriptions": 1928, "Questions": 244, "Answers": 351, "Pyongs": 41}, "roles":

["Editor", "Contributor"], "followers_num": 146, "following_num": 123}
{"url_name": "YABOYNICKY", "iq": 62747, "stats": {"Annotations"

There are some limitations given this information:

File	Apparent Structure	Implications
annotation_info.json	Annotation metadata, lit of contribution stats, list of edit objects, long HTML text	Most complicated file, extract select metadata and relationships in batches, avoid retaining text unless necessary
artist_info.json	Artist profiles, “roles” array, extensive “songs” array	Keep song relationships separate from profile fields, verify what an “artist” record represents
song_info.json	Song metadata, “tags” array, annotation is “null”	Useful for song-to-tag relationships, tags include genres and other labels
user_info.json	User level fields, nested stats, “roles” array	Natural starting point for user attributes, observed follower counts are not an edge list

- song_info includes non-genre tags and so an instance like “Memes” should not be interpreted as such
- artist_info has various fields so the file name itself does not clearly establish the entity it covers (though the original README in genius-expertise does clarify this)
- annotation prefix did not show a song ID; need to inspect the complete record’s field names before determining how to connect annotations to songs

Data Inspection & Preliminary Diagnostics

```
# confirming format with extended but limited preview
check_json_prefix <- function(path, max_bytes = 262144L) {
  con <- file(path, open = "rb")
  on.exit(close(con))
  # storing data into variable
  bytes <- readBin(con, what = "raw", n = max_bytes)
  # determining newline positions
  newlines <- which(bytes == as.raw(10))
  # providing early exit case
  if (!length(newlines)) {
    return(data.frame(
```

```
      file = basename(path),
      complete_lines = 0L,
      valid_json_lines = NA_integer_
    ))
  }
  # normalizing text for easier parsing
  end <- tail(newlines, 1)
  text <- rawToChar(bytes[seq_len(end)])
  lines <- strsplit(text, "\n", fixed = TRUE)[[1]]
  lines <- lines[nzchar(trimws(lines))]
  # valid data check
  valid <- vapply(lines, jsonlite::validate, logical(1))
  # providing one row summary table
  data.frame(
    file = basename(path),
    complete_lines = length(lines),
    valid_json_lines = sum(valid)
  )
}
json_files <- list.files(
  data_dir,
  pattern = "\\.json$",
  full.names = TRUE,
  ignore.case = TRUE
)
do.call(rbind, lapply(json_files, check_json_prefix))
```

	file	complete_lines	valid_json_lines
1	annotation_info.json	17	17
2	artist_info.json	8	8
3	song_info.json	1028	1028
4	user_info.json	1179	1179

This table confirms every line check is valid. We can inspect the json specific information from here.

```
# limiting preview of text file
read_record_preview <- function(path, n = 5L) {
  con <- file(path, open = "rt", encoding = "UTF-8")
  on.exit(close(con))
  # convering json to nested r list
  lines <- readLines(con, n = n, warn = FALSE)
  lapply(lines, function(line) {
    jsonlite::fromJSON(line, simplifyVector = FALSE)
  })
}
# storing user info preview
users_preview <- read_record_preview(
  file.path(data_dir, "user_info.json")
)
```

```
# displaying user info preview
names(users_preview[[1]])
```

```
[1] "url_name"      "iq"            "stats"         "roles"
[5] "followers_num" "following_num"
```

```
str(users_preview[[1]], max.level = 2)
```

```
List of 6
 $ url_name     : chr "QueenOfCoplaints"
 $ iq           : int 446264
 $ stats        :List of 6
  ..$ Annotations   : int 5072
  ..$ Suggestions   : int 487
  ..$ Transcriptions: int 1808
  ..$ Questions     : int 13
  ..$ Answers       : int 24
  ..$ Pyongs        : int 5601
 $ roles        :List of 2
  ..$ : chr "Editor"
  ..$ : chr "Contributor"
 $ followers_num: int 600
 $ following_num: int 1
```

Candidate relationships:

- contribution_stats$name -> user_info$url_name
- annotation$song -> song_info$url_name
- song_info$tags -> genre-related categories

```
# limiting preview of text file
annotation_preview <- read_record_preview(
  file.path(data_dir, "annotation_info.json"),
  n = 1L
)
names(annotation_preview[[1]])
```

```
 [1] "time"              "type"              "votes"
 [4] "pyongs"            "acceptor"          "url"
 [7] "contribution_stats" "edits_lst"        "song"
[10] "lyrics"
```

```
# inspecting limited contributor structure
str(
  annotation_preview[[1]]$contribution_stats,
  max.level = 2,
```

```
  list.len = 3
)
```

```
List of 2
 $ :List of 2
  ..$ name        : chr "PiecesOfAMan"
  ..$ contribution: num 73.7
 $ :List of 2
  ..$ name        : chr "Incilin"
  ..$ contribution: num 26.3
```

```
# early stopping message
if (!dir.exists(data_dir)) {
  stop("Dataset folder not found. Check the data_dir path.")
}

# reading limited JSON files
read_records <- function(path, n = 25L) {
  if (!file.exists(path)) {
    stop("File not found: ", path)
  }
  # opening connection
  con <- file(path, open = "rt", encoding = "UTF-8")
  # closing connection
  on.exit(close(con))
  # reading requested number of lines
  lines <- readLines(con, n = n, warn = FALSE)
  # convering lines into separate r lists
  lapply(lines, function(line) {
    jsonlite::fromJSON(line, simplifyVector = FALSE)
  })
}
# reading 25 annotations for testing
annotations_test <- read_records(
  file.path(data_dir, "annotation_info.json"),
  n = 25L
)
# inspecting song field
length(annotations_test)
```

```
[1] 25
```

```
str(annotations_test[[1]]$song)
```

```
 chr "Kendrick-lamar-swimming-pools-drank-lyrics"
```

```
# inspecting contributor structure
str(
  annotations_test[[1]]$contribution_stats,
```

```
  max.level = 2,
  list.len = 3
)
```

```
List of 2
 $ :List of 2
  ..$ name        : chr "PiecesOfAMan"
  ..$ contribution: num 73.7
 $ :List of 2
  ..$ name        : chr "Incilin"
  ..$ contribution: num 26.3
```

This output confirms the test records have an expected structure for its extraction. Now, I will test out contributor to annotation relationships.

```
# returning single values as text
get_text <- function(x) {
  if (is.null(x) || length(x) != 1L || !is.atomic(x)) {
    return(NA_character_)
  }
  as.character(x)
}

# returning single values as numeric
get_number <- function(x) {
  if (is.null(x) || length(x) != 1L || !is.atomic(x)) {
    return(NA_real_)
  }
  # otherwise marked as NA
  suppressWarnings(as.numeric(x))
}
```

```
# converting single annotation to small table of contributors
extract_contributors <- function(annotation) {
  contributors <- annotation$contribution_stats

  # returning NULL if no listed contributors
  if (is.null(contributors) || length(contributors) == 0L) {
    return(NULL)
  }
  # creating single row for each listed contributor to an annotation
  rows <- lapply(contributors, function(person) {
    data.frame(
      annotation_url = get_text(annotation$url),
      song = get_text(annotation$song),
      user = get_text(person$name),

      # preserving original contribution value
      contribution = get_number(person$contribution),
```

```
      stringsAsFactors = FALSE
    )
  })

  # combining contributors for single annotation into a table
  do.call(rbind, rows)
}
# applying extraction across 25 test records
contributor_song_test <- data.table::rbindlist(
  lapply(annotations_test, extract_contributors),
  use.names = TRUE,
  fill = TRUE
)
# checking extracted table results
head(contributor_song_test)
```

```
              annotation_url                                      song
                      <char>                                    <char>
1: https://genius.com/2739115 Kendrick-lamar-swimming-pools-drank-lyrics
2: https://genius.com/2739115 Kendrick-lamar-swimming-pools-drank-lyrics
3: https://genius.com/1072360 Kendrick-lamar-swimming-pools-drank-lyrics
4: https://genius.com/1072360 Kendrick-lamar-swimming-pools-drank-lyrics
5: https://genius.com/1072360 Kendrick-lamar-swimming-pools-drank-lyrics
6:  https://genius.com/905517 Kendrick-lamar-swimming-pools-drank-lyrics
                   user contribution
                  <char>        <num>
1:          PiecesOfAMan        73.73
2:               Incilin        26.27
3:               Incilin        75.00
4:           Noctoural88        12.50
5: BoredBorderlineGenius        12.50
6:              Haifisch        31.82
```

```
str(contributor_song_test)
```

```
Classes 'data.table' and 'data.frame':  136 obs. of  4 variables:
 $ annotation_url: chr  "https://genius.com/2739115" "https://genius.com/2739115"
"https://genius.com/1072360" "https://genius.com/1072360" ...
 $ song          : chr  "Kendrick-lamar-swimming-pools-drank-lyrics" "Kendrick-lamar-
swimming-pools-drank-lyrics" "Kendrick-lamar-swimming-pools-drank-lyrics" "Kendrick-
lamar-swimming-pools-drank-lyrics" ...
 $ user          : chr  "PiecesOfAMan" "Incilin" "Incilin" "Noctoural88" ...
 $ contribution  : num  73.7 26.3 75 12.5 12.5 ...
 - attr(*, ".internal.selfref")=<pointer: 0x10391bc10>
```

```
# confirming expected columns were produced
expected_columns <- c(
  "annotation_url", "song", "user", "contribution"
)
```

```
if (!all(expected_columns %in% names(contributor_song_test))) {
  stop("No complete extraction table was produced. Inspect the test records.")
}
# counting missing values in extracted columns
colSums(is.na(contributor_song_test))
```

```
annotation_url           song           user   contribution
             0              0              0              0
```

```
# summarzing test
data.frame(
  annotations_read = length(annotations_test),
  contributor_rows = nrow(contributor_song_test),
  unique_users = length(unique(
    contributor_song_test$user[!is.na(contributor_song_test$user)]
  )),
  unique_songs = length(unique(
    contributor_song_test$song[!is.na(contributor_song_test$song)]
  ))
)
```

```
  annotations_read contributor_rows unique_users unique_songs
1               25              136           99            3
```

```
summary(contributor_song_test$contribution)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  0.450   2.745   7.800  15.442  19.517  87.140
```

This test converted 25 annotation records into a table of 136 contributor to annotation relationships, retaining the annotation ID, song ID, contributor name, and contribution value. After inspecting the extracted structure, it is confirmed that all four columns had no missing values, there are 99 different users and 3 different songs, and contribution values range from 0.45 to 87.14 and a median of 7.8.

Now, we can try to establish that identifiers match across files or even support conclusions for the broader network.

```
# reading json files, keeping only identifiers
read_ids <- function(path) {
  ids <- character()
  con <- file(path, open = "rt", encoding = "UTF-8")
  on.exit(close(con))
  # running limited json lines through
  jsonlite::stream_in(
    con,
    pagesize = 500,
    verbose = FALSE,
    simplifyVector = FALSE,
    # processing each batch iteratively
```

```
    handler = function(batch) {
      batch_ids <- vapply(batch, function(record) {
        id <- record$url_name
        if (is.null(id) || length(id) != 1L) {
          return(NA_character_)
        }
        as.character(id)
      }, character(1))
      ids <<- c(ids, batch_ids)
    }
  )
  # keeping relevant identifiers
  unique(ids[!is.na(ids) & nzchar(ids)])
}
```

```
# scanning the two reference files
song_ids <- read_ids(file.path(data_dir, "song_info.json"))
user_ids <- read_ids(file.path(data_dir, "user_info.json"))
# checking distinct identifiers
test_songs <- unique(contributor_song_test$song)
test_users <- unique(contributor_song_test$user)
# reporting how many identifiers have exact match
match_summary <- data.frame(
  identifier = c("Songs", "Users"),
  checked = c(length(test_songs), length(test_users)),
  matched = c(
    sum(test_songs %in% song_ids),
    sum(test_users %in% user_ids)
  )
)
match_summary$unmatched <- with(match_summary, checked - matched)
match_summary
```

```
  identifier checked matched unmatched
1      Songs       3       3         0
2      Users      99      83        16
```

```
# showing any unmatched identifiers
setdiff(test_songs, song_ids)
```

```
character(0)
```

```
setdiff(test_users, user_ids)
```

```
 [1] "Crusade00"          "K-NOS"              "yellowcaution"
 [4] "benru2219"          "DrLanz"             "JackSwagger"
 [7] "JeroenVerschooren"  "StarlaA"            "PatrickMacKay"
[10] "dukem"              "anotherotherperson" "Taav"
```

```
[13] "downandout"         "holaaurora"         "salece"
[16] "cdevon87"
```

```
# preserving all contributor rows
contributor_song_test$user_profile_match <-
  contributor_song_test$user %in% user_ids
contributor_song_test$song_match <-
  contributor_song_test$song %in% song_ids
# summarizing contributors by provile availability
table(contributor_song_test$user_profile_match)
```

```
FALSE  TRUE
   16   120
```

All 3 distinct song identifiers matched for song metadata. 83/99 contributors matched between attribute files while 16 did not. Of the 136 total contributor to annotation records, 120 matched and again, 16 did not. These unmatched records were retaining and flagged for analytical consideration.

Analyzing Data & Attributes

I realized above that map_artist_names.csv is interpreting the first row, Black Thought, as the header instead of being the first instance. Otherwise, we can now dissect the information knowing that from a limited sample size that this network has a consistent connection basis.

```
# reading mapping without header issue
artist_map <- data.table::fread(
  file.path(data_dir, "map_artist_names.csv"), header = FALSE,
  col.names = c("name_original", "name_mapped"), colClasses = "character"
)
# confirm this instance has been resolved
head(artist_map)
```

```
     name_original       name_mapped
             <char>            <char>
1:    blackthought     Black-thought
2:          PushaT           Pusha-t
3:      RobMarkman       Rob-markman
4: ChanceTheRapper Chance-the-rapper
5:   KendrickLamar    Kendrick-lamar
6:    Savan_DePaul      Savan-depaul
```

```
# necessary imports
library(data.table)
```

```
Attaching package: 'data.table'
```

```
The following object is masked from 'package:base':

    %notin%
```

```
library(igraph)
```

```
Attaching package: 'igraph'

The following objects are masked from 'package:stats':

    decompose, spectrum

The following object is masked from 'package:base':

    union
```

```
# importing complete follower edge list
follows <- fread(
  file.path(data_dir, "follows.csv"),
  select = c("follower", "following"),
  colClasses = "character"
)
# determining untenable endpoints while maintaining normalization
bad_endpoint <- with(
  follows, is.na(follower) | is.na(following) |
    trimws(follower) == "" | trimws(following) == ""
)
```

```
# maintaining usable edges in a separate object
edges <- follows[!bad_endpoint]
# repoting issues before error handling
data.frame(
  imported_rows = nrow(follows),
  missing_or_blank_endpoints = sum(bad_endpoint),
  self_loop_rows = sum(edges$follower == edges$following),
  duplicate_rows = sum(duplicated(edges))
)
```

```
  imported_rows missing_or_blank_endpoints self_loop_rows duplicate_rows
1       1778595                         0           1219              0
```

```
# constructing directed graph
g_directed <- graph_from_data_frame(
  edges[, .(follower, following)],
  directed = TRUE
)
# first network summary.
data.frame(
```

```
  nodes = vcount(g_directed), edges = ecount(g_directed),
  directed = is_directed(g_directed), simple = is_simple(g_directed)
)
```

```
   nodes   edges directed simple
1 780844 1778595     TRUE  FALSE
```

These are the baseline graph construction results. Also note that directed = TRUE so which suggests that if follower i follows j, that does not inherently mean j follows i, and simple = FALSE so self loops are present throughout the graph.

Output	Interpretation
1,778,595 imported rows	Recorded follower relationships before cleaning
0 missing / blank endpoints	Every row has both endpoint identifiers
1,219 self-loops	Rows where follower and following match
0 duplicate rows	No repeated follower-following pairs
780,844 nodes	Distinct identifiers appearing as edge endpoints

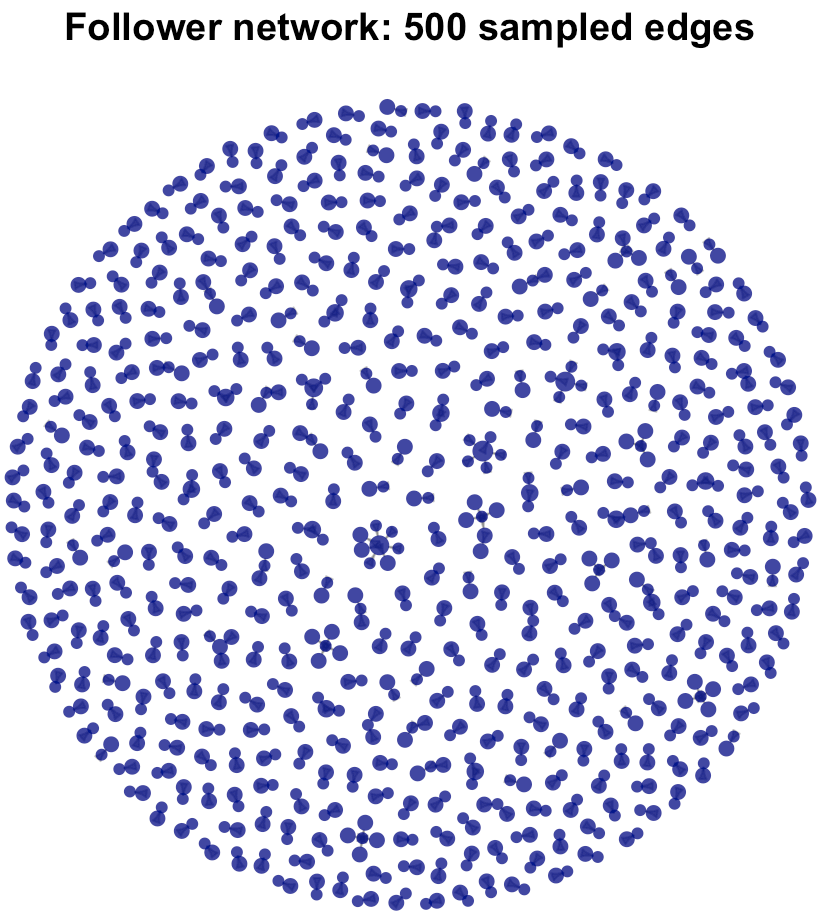
It seems reasonable to remove self loops given we are specifically looking for relationships between different users and not that same ones. This does impact network characteristics and even some statistics such as density. Self loops are about 0.07% of the edges which may have impacts on lower degree users. All vertices are retained and this is overall a small amount of information discarded.

```
# creating graph with relationships between different users
g_directed_clean <- igraph::simplify(
  g_directed, remove.multiple = FALSE, remove.loops = TRUE
)
# verifying the clean graph size and structure
data.frame(
  nodes = igraph::vcount(g_directed_clean),
  edges = igraph::ecount(g_directed_clean),
  simple = igraph::is_simple(g_directed_clean)
)
```

```
   nodes   edges simple
1 780844 1777376   TRUE
```

```
# sampling 500 edges
set.seed(585)
g_plot <- igraph::subgraph_from_edges(
  g_directed_clean,
  eids = sample.int(igraph::ecount(g_directed_clean), 500),
  delete.vertices = TRUE
)
```

```
# calculating positions for sampled graph
lay <- igraph::layout_with_fr(g_plot)
old_mar <- par("mar")
par(mar = c(0, 0, 2, 0))
# plotting graph with various characteristics included
plot(
  g_plot,
  layout = lay,
  vertex.label = NA,
  vertex.size = 3 + sqrt(igraph::degree(g_plot, mode = "in")),
  vertex.color = adjustcolor("darkblue", 0.8),
  vertex.frame.color = NA,
  edge.arrow.size = 0.3,
  edge.width = 0.7,
  edge.color = adjustcolor("grey40", 0.4),
  main = "Follower network: 500 sampled edges"
)
```




```
par(mar = old_mar)
```

```
# using cleaned graph for baseline statistics
g <- g_directed_clean
# establishing in degree and out degree
```

```
d_in <- igraph::degree(g, mode = "in")
d_out <- igraph::degree(g, mode = "out")
```

```
# summarizing various attributes
baseline <- data.frame(
  nodes = igraph::vcount(g), edges = igraph::ecount(g),
  isolates = sum(d_in + d_out == 0), mean_in_degree = mean(d_in),
  median_in_degree = median(d_in), mean_out_degree = mean(d_out),
  median_out_degree = median(d_out), density = igraph::edge_density(g, loops = FALSE)
)
baseline
```

```
   nodes   edges isolates mean_in_degree median_in_degree mean_out_degree
1 780844 1777376        1       2.276224                1        2.276224
  median_out_degree      density
1                 0 2.915086e-06
```

```
# examining variations
summary(d_in)
```

```
    Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
   0.000    1.000    1.000    2.276    1.000 7415.000
```

```
summary(d_out)
```

```
     Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
    0.000     0.000     0.000     2.276     0.000 13880.000
```